

Apriori Algorithm

Association Rule Mining with a Worked Example

Ananda Rimal

Machine Learning Lecture

August 15, 2026

What We Will Learn Today

- ① What is the Apriori algorithm and why do we use it
- ② Real life example: Market Basket Analysis
- ③ Important terms: itemset, support, confidence, lift
- ④ Architecture and steps of the algorithm
- ⑤ Full numerical example solved step by step
- ⑥ Generating association rules from the result

What is the Apriori Algorithm?

- Apriori is a classic algorithm for **Association Rule Mining**.
- It finds **items that frequently appear together** in a large database of transactions.
- Given shopping data, it can discover patterns like:

**“Customers who buy Bread and Milk
often also buy Diapers”**

- It was proposed by **Agrawal and Srikant in 1994**.
- It is an **unsupervised learning** method. There is no target label. We only search for hidden patterns.

Why is it Used? (Applications)

- **Market Basket Analysis:** which products sell together in a supermarket
- **Product placement:** keep related items close on shelves
- **Recommendation systems:** “Customers who bought this also bought...”
- **Medical diagnosis:** symptoms that occur together
- **Web usage mining:** pages visited together

Bread, Milk, Butter

Basket 1

Bread, Milk, Jam

Basket 2

Bread, Milk, Eggs

Basket 3

Pattern:
Bread and Milk
bought together

Famous Real Life Example: Beer and Diapers

- A store in the USA analyzed its sales data and found a strange pattern: **Beer and Diapers were often bought together on Friday evenings.**
- Reason: Young fathers buying diapers for babies also picked up beer for the weekend.
- Action: The store placed beer near diapers and **sales increased.**



Key Idea

Hidden patterns in transaction data can directly increase business profit.

Related Terms 1: Transaction and Itemset

- **Item:** a single product. Example: Bread
- **Transaction:** one customer bill, the set of items bought together. Example: $T1 = \{\text{Bread, Milk}\}$
- **Itemset:** any collection of one or more items. Example: $\{\text{Bread, Milk}\}$
- **k-itemset:** an itemset with k items. $\{\text{Bread, Milk, Beer}\}$ is a 3-itemset.

TID	Items bought
T1	Bread, Milk
T2	Bread, Diaper, Beer

- **Frequent itemset:** an itemset that appears in the data at least a minimum number of times (minimum support).

Related Terms 2: Support

Support

Support tells us **how popular an itemset is** in the whole data.

$$\text{Support}(A) = \frac{\text{Number of transactions containing } A}{\text{Total number of transactions}}$$

- Example: Bread appears in 4 out of 5 transactions.

$$\text{Support}(\text{Bread}) = \frac{4}{5} = 0.8 = 80\%$$

- **Minimum support (min_sup)**: a threshold we choose. Itemsets below it are thrown away as “rare”.
- Sometimes we use **support count** (just the number, like 4) instead of the fraction.

Related Terms 3: Confidence

Confidence

Confidence measures **how reliable a rule** $A \Rightarrow B$ is.

$$\text{Confidence}(A \Rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

“Out of all customers who bought A, how many also bought B?”

- Example: 4 people bought Diaper, 3 of them also bought Beer.

$$\text{Confidence}(\text{Diaper} \Rightarrow \text{Beer}) = \frac{3}{4} = 75\%$$

- **Minimum confidence (min_conf):** rules below this threshold are rejected.

Related Terms 4: Lift

Lift

Lift checks whether A and B are **really related** or appear together just by chance.

$$\text{Lift}(A \Rightarrow B) = \frac{\text{Confidence}(A \Rightarrow B)}{\text{Support}(B)}$$

Lift value	Meaning
Lift > 1	A and B are positively related (good rule)
Lift = 1	A and B are independent (no relation)
Lift < 1	A and B are negatively related

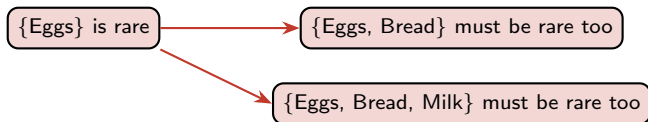
- Example: if Confidence = 100% and Support(B) = 80%, then Lift = $1.0/0.8 = 1.25 > 1$, so the rule is meaningful.

Related Terms 5: The Apriori Property

Apriori Property (the heart of the algorithm)

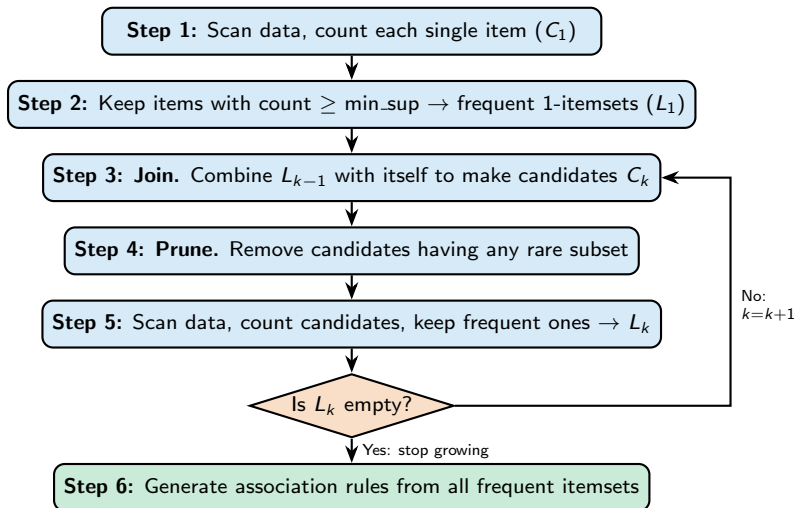
All subsets of a frequent itemset must also be frequent.

In reverse: if an itemset is NOT frequent, none of its supersets can be frequent.



- This lets us **prune (cut away)** huge numbers of candidates early.
- That is why the algorithm is fast compared to checking every possible itemset.

Architecture of the Apriori Algorithm



Two Key Operations: Join and Prune

Join Step

Make bigger candidates from smaller frequent itemsets.

- $\{\text{Bread, Milk}\} + \{\text{Bread, Diaper}\}$
- $\rightarrow \{\text{Bread, Milk, Diaper}\}$

Two $(k-1)$ -itemsets join only if their first $k-2$ items are the same.

Prune Step

Before counting, check every $(k-1)$ subset of the candidate.

- If ANY subset is not in L_{k-1} , delete the candidate immediately.
- Uses the Apriori property. Saves a lot of counting work.

The algorithm repeats **Join** $\rightarrow$ **Prune** $\rightarrow$ **Count** level by level:
 $L_1 \rightarrow L_2 \rightarrow L_3 \dots$ until no new frequent itemset is found.

Numerical Example: The Dataset

TID	Items bought
T1	Bread, Milk
T2	Bread, Diaper, Beer, Eggs
T3	Milk, Diaper, Beer, Cola
T4	Bread, Milk, Diaper, Beer
T5	Bread, Milk, Diaper, Cola

Given conditions

- Total transactions = 5
- **Minimum support count = 2** (that is $2/5 = 40\%$)
- **Minimum confidence = 60%**

Task: find all frequent itemsets, then generate strong rules.

Iteration 1: $C_1 \rightarrow L_1$ (Single Items)

Step 1: Scan all 5 transactions and count every item (C_1):

Itemset	Count	Frequent? (count ≥ 2)
{Bread}	4	Yes
{Milk}	4	Yes
{Diaper}	4	Yes
{Beer}	3	Yes
{Cola}	2	Yes
{Eggs}	1	No, remove

Result:

$$L_1 = \{\text{Bread}\}, \{\text{Milk}\}, \{\text{Diaper}\}, \{\text{Beer}\}, \{\text{Cola}\}$$

Eggs appears only once, so by the Apriori property no itemset containing Eggs can ever be frequent.

Iteration 2: $C_2 \rightarrow L_2$ (Pairs)

Join: make all pairs from L_1 , then count them in the data:

Pair	Count	Pair	Count
{Bread, Milk}	3 ✓	{Milk, Beer}	2 ✓
{Bread, Diaper}	3 ✓	{Milk, Cola}	2 ✓
{Bread, Beer}	2 ✓	{Diaper, Beer}	3 ✓
{Bread, Cola}	1 ✗	{Diaper, Cola}	2 ✓
{Milk, Diaper}	3 ✓	{Beer, Cola}	1 ✗

Remove {Bread, Cola} and {Beer, Cola} because count < 2 .

$$L_2 = \{B, M\}, \{B, D\}, \{B, Be\}, \{M, D\}, \{M, Be\}, \{M, C\}, \{D, Be\}, \{D, C\}$$

(B = Bread, M = Milk, D = Diaper, Be = Beer, C = Cola)

Iteration 3: $C_3 \rightarrow L_3$ (Triplets)

Join L_2 with L_2 , then prune using the Apriori property:

Candidate	Subset check	Count
{Bread, Milk, Diaper}	all pairs frequent	2 ✓
{Bread, Milk, Beer}	all pairs frequent	1 ✗
{Bread, Diaper, Beer}	all pairs frequent	2 ✓
{Milk, Diaper, Beer}	all pairs frequent	2 ✓
{Milk, Diaper, Cola}	all pairs frequent	2 ✓
{Bread, Milk, Cola}	{Bread, Cola} not in L_2 : pruned	-
{Milk, Beer, Cola}	{Beer, Cola} not in L_2 : pruned	-

$$L_3 = \{B, M, D\}, \{B, D, Be\}, \{M, D, Be\}, \{M, D, C\}$$

Iteration 4: every 4-item candidate (like {Bread, Milk, Diaper, Beer}) contains a rare subset ({Bread, Milk, Beer}), so C_4 becomes empty. **The algorithm stops.**

Generating Rules from {Milk, Diaper, Beer}

Take the frequent itemset {Milk, Diaper, Beer} with support count 2.
Split it into all possible rules and compute confidence:

Rule	Confidence	$\geq 60\%$?
Milk, Diaper $\Rightarrow$ Beer	$2/3 = 66.7\%$	Accept
Milk, Beer $\Rightarrow$ Diaper	$2/2 = 100\%$	Accept
Diaper, Beer $\Rightarrow$ Milk	$2/3 = 66.7\%$	Accept
Milk $\Rightarrow$ Diaper, Beer	$2/4 = 50\%$	Reject
Diaper $\Rightarrow$ Milk, Beer	$2/4 = 50\%$	Reject
Beer $\Rightarrow$ Milk, Diaper	$2/3 = 66.7\%$	Accept

Example calculation:

$$\text{Conf}(\text{Milk, Beer} \Rightarrow \text{Diaper}) = \frac{\text{count}\{M, D, Be\}}{\text{count}\{M, Be\}} = \frac{2}{2} = 100\%$$

Checking a Rule with Lift

Take the strongest rule: **Milk, Beer** $\Rightarrow$ **Diaper** (confidence 100%).

$$\text{Support}(\text{Diaper}) = \frac{4}{5} = 0.8$$

$$\text{Lift} = \frac{\text{Confidence}}{\text{Support}(\text{Diaper})} = \frac{1.0}{0.8} = 1.25$$

- Lift = 1.25 > 1, so buying {Milk, Beer} truly **increases** the chance of buying Diaper.
- The relationship is real, not just a coincidence.

Final answer of the example

Strong rules include: {Milk, Beer} $\Rightarrow$ Diaper (100%), {Milk, Diaper} $\Rightarrow$ Beer (66.7%), {Diaper, Beer} $\Rightarrow$ Milk (66.7%), Beer $\Rightarrow$ {Milk, Diaper} (66.7%)

Advantages and Limitations

Advantages

- Simple and easy to understand
- Results (rules) are easy to explain to business people
- Pruning removes many useless candidates
- Works on any transaction style data

Limitations

- Scans the whole database many times (slow for big data)
- Can still generate many candidates when min_sup is low
- Faster alternatives exist, like **FP-Growth**

Summary

- 1 Apriori finds **frequent itemsets** and **association rules** from transaction data.
- 2 Key terms: **support** (popularity), **confidence** (reliability), **lift** (real relation or chance).
- 3 Core trick: the **Apriori property**. If an itemset is rare, all of its supersets are rare.
- 4 Process: $C_1 \rightarrow L_1 \rightarrow C_2 \rightarrow L_2 \rightarrow C_3 \rightarrow L_3 \dots$ using **Join, Prune, Count** until nothing new is frequent.
- 5 From frequent itemsets, keep only rules with confidence $\geq \text{min_conf}$, then verify with lift.

Practice: redo the example with minimum support count = 3.
Which itemsets survive?